

St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026

Class: BE-ITA/B, Semester: VIII

Subject: BlockChain Lab

Experiment – 6

1. **Aim:** Case study on Hyperledger blockchain.
2. **Lab outcome:** After performing the experiment, the students will be able to **comprehend** the working of Hyperledger blockchain. (PO1, PSO1, BL2)

3. **Prerequisite:**

Fundamental knowledge of blockchain

4. **Requirements:** The following are the requirements –
PC and Internet.

5. **Pre-Experiment Theory:**

What is Hyperledger?

Hyperledger is an open-source project created to support the development of blockchain-based distributed ledgers. Hyperledger consists of a collaborative effort to create the needed frameworks, standards, tools and libraries to build blockchains and related applications. Hyperledger was created by the Linux Foundation in 2016. Hyperledger Fabric, an open- source project from the Linux Foundation, is the modular blockchain framework and standard for enterprise blockchain platforms. Intended as a foundation for developing enterprise-grade applications and industry solutions, the open, modular architecture uses plug-and-play components to accommodate a wide range of use cases.

Hyperledger Composer is a set of collaboration tools for building blockchain business networks that make it simple and fast for business owners and developers to create smart contracts and blockchain applications to solve business problems. Built with JavaScript, leveraging modern tools including node.js, npm, CLI and popular editors, Composer offers business centric abstractions as well as sample apps with easy to test devops processes to create robust blockchain solutions that drive alignment across business requirements with technical development.

A smart contract in Hyperledger Fabric is a program, called chaincode which is written in Go, JavaScript, and other programming languages such as Java that implement a prescribed interface.

6. **Laboratory Exercise**

7. **Steps to be implemented.**

Select any one research paper regarding case study on Hyperledger Blockchain, study it and write report based on your studies.

8. **Program Code**

Write any one sample chaincode in Golang for hyperledger

9. **Post Experimental Exercise-**

10. **Questions:**

How a chaincode is implemented on Hyperledger blockchain?

11. **Results/Observations/Program output:**

Present the program input/output results if any and comment on the same.

12. Conclusion

- a. Write what was performed in the experiment.
- b. Write which tools you used to perform the experiment
- c. Write what you inferred from the output obtained.

13. References:

- [1] <https://www.hyperledger.org/learn/case-studies>
- [2] <https://hyperledger-fabric.readthedocs.io/en/release-2.2/whatis.h>
- [3] Hyperledger Fabric In-Depth: Learn, Build and Deploy Blockchain applications Using Hyperledger Fabric, Ashwani Kumar, BPB publications

Steps to be implemented.

Select any one research paper regarding case study on Hyperledger Blockchain, study it and write a report based on your studies.

Title: Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains

Authors: Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Stathakopoulou, Marko Vukolić, Sharon Weed Cocco, and Jason Yellick.

SUMMARY:

Hyperledger Fabric represents a structural breakthrough in blockchain architecture by redefining how distributed ledgers process transactions in enterprise environments. Unlike public blockchains such as Bitcoin and Ethereum, which operate in open, adversarial settings and depend on cryptocurrency-based incentive mechanisms, Fabric is designed for permissioned networks where participants are known and authenticated. Its importance lies not merely in improved performance, but in a fundamental architectural shift that resolves core inefficiencies present in earlier blockchain systems.

Traditional blockchains follow an order–execute model in which transactions are first agreed upon through consensus and then sequentially executed by every node. While this guarantees consistency, it creates performance bottlenecks, strict determinism requirements, and confidentiality limitations because every peer must execute and observe every transaction. Such a model scales poorly and is unsuitable for regulated industries requiring privacy controls and high throughput. Fabric addresses these structural constraints through its execute–order–validate paradigm, which decouples transaction execution from consensus and final commitment.

Beyond architectural innovation, Fabric introduces features that enhance its novelty. Smart contracts, known as chaincode, can be written in general-purpose languages such as Go, Java, and Node.js, eliminating reliance on domain-specific virtual machines. It does not require a native cryptocurrency or gas mechanism, removing economic volatility and simplifying governance. Endorsement policies allow applications to define custom trust rules independent of consensus, while channels enable private sub-ledgers for selective data visibility. Performance benchmarks demonstrate throughput exceeding 3500 transactions per second with sub-second latency, significantly outperforming many public chains.

The real-world significance of Hyperledger Fabric lies in proving that blockchain technology can function as a modular distributed system optimized for business collaboration rather than speculative token ecosystems. Its deployment in supply chains, trade finance, identity management, and digital asset settlement demonstrates practical viability.

Code:

```
package main

import (
    "encoding/json"
    "fmt"

    "github.com/hyperledger/fabric-contract-api-go/contractapi"
)

// SmartContract provides functions for managing an Asset
type SmartContract struct {
    contractapi.Contract
}

// Asset structure
type Asset struct {
    ID   string `json:"id"`
    Owner string `json:"owner"`
    Value int   `json:"value"`
}

// CreateAsset adds a new asset to the ledger
func (s *SmartContract) CreateAsset(ctx contractapi.TransactionContextInterface, id string,
owner string, value int) error {

    exists, err := s.AssetExists(ctx, id)
    if err != nil {
        return err
    }
    if exists {
        return fmt.Errorf("asset %s already exists", id)
    }

    asset := Asset{
        ID:   id,
        Owner: owner,
        Value: value,
    }

    assetJSON, err := json.Marshal(asset)
    if err != nil {
        return err
    }
```

```
        return ctx.GetStub().PutState(id, assetJSON)
    }

// ReadAsset returns an asset from ledger
func (s *SmartContract) ReadAsset(ctx contractapi.TransactionContextInterface, id string)
(*Asset, error) {

    assetJSON, err := ctx.GetStub().GetState(id)
    if err != nil {
        return nil, fmt.Errorf("failed to read from world state: %v", err)
    }
    if assetJSON == nil {
        return nil, fmt.Errorf("asset %s does not exist", id)
    }

    var asset Asset
    err = json.Unmarshal(assetJSON, &asset)
    if err != nil {
        return nil, err
    }

    return &asset, nil
}

// TransferAsset changes the owner of an existing asset
func (s *SmartContract) TransferAsset(ctx contractapi.TransactionContextInterface, id string,
newOwner string) error {

    asset, err := s.ReadAsset(ctx, id)
    if err != nil {
        return err
    }

    asset.Owner = newOwner

    assetJSON, err := json.Marshal(asset)
    if err != nil {
        return err
    }

    return ctx.GetStub().PutState(id, assetJSON)
}

// AssetExists checks if asset exists
func (s *SmartContract) AssetExists(ctx contractapi.TransactionContextInterface, id string)
(bool, error) {

    assetJSON, err := ctx.GetStub().GetState(id)
    if err != nil {
        return false, err
    }
}
```

```

    return assetJSON != nil, nil
}

func main() {
    chaincode, err := contractapi.NewChaincode(new(SmartContract))
    if err != nil {
        fmt.Printf("Error creating chaincode: %v", err)
        return
    }

    if err := chaincode.Start(); err != nil {
        fmt.Printf("Error starting chaincode: %v", err)
    }
}

```

```

tannmayy14@TanmayLaptop:~/projects$ git clone https://github.com/hyperledger/fabric-samples.git
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15522, done.
remote: Counting objects: 100% (217/217), done.
remote: Compressing objects: 100% (147/147), done.
remote: Total 15522 (delta 108), reused 95 (delta 66), pack-reused 15305 (from 4)
Receiving objects: 100% (15522/15522), 24.29 MiB | 4.58 MiB/s, done.
Resolving deltas: 100% (8716/8716), done.

```

Downloading the fabric binaries

```

Status: Downloaded newer image for hyperledger/fabric-ca:1.5.17
docker.io/hyperledger/fabric-ca:1.5.17
==> List out hyperledger docker images
hyperledger/fabric-peer      2.5             c48323777139    6 days ago      230MB
hyperledger/fabric-peer      2.5.15          c48323777139    6 days ago      230MB
hyperledger/fabric-peer      latest          c48323777139    6 days ago      230MB
hyperledger/fabric-orderer   2.5             72697d099f45    6 days ago      178MB
hyperledger/fabric-orderer   2.5.15          72697d099f45    6 days ago      178MB
hyperledger/fabric-orderer   latest          72697d099f45    6 days ago      178MB
hyperledger/fabric-ccenv     2.5             29cce493a269    6 days ago      1GB
hyperledger/fabric-ccenv     2.5.15          29cce493a269    6 days ago      1GB
hyperledger/fabric-ccenv     latest          29cce493a269    6 days ago      1GB
hyperledger/fabric-baseos    2.5             c8b3d419307a    6 days ago      250MB
hyperledger/fabric-baseos    2.5.15          c8b3d419307a    6 days ago      250MB
hyperledger/fabric-baseos    latest          c8b3d419307a    6 days ago      250MB
hyperledger/fabric-ca        1.5             fac0c4a70afc    3 weeks ago     381MB
hyperledger/fabric-ca        1.5.17          fac0c4a70afc    3 weeks ago     381MB
hyperledger/fabric-ca        latest          fac0c4a70afc    3 weeks ago     381MB

```

Pulling docker images

```

tannmayy14@TanmayLaptop:~/projects/fabric-samples/test-network$ ./network.sh up
Using docker and docker-compose
Starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb' with crypto from 'cryptogen'
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/tannmayy14/projects/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[*] Running 7/7
✓Network fabric.test Created 0.1s
✓Volume "compose_peer0_org1.example.com" Created 0.0s
✓Volume "compose_peer0_org2.example.com" Created 0.0s
✓Volume "compose_orderer.example.com" Created 0.0s
✓Container orderer.example.com Started 1.6s
✓Container peer0.org1.example.com Started 1.5s
✓Container peer0.org2.example.com Started 1.3s
CONTAINER ID   IMAGE                                NAMES                COMMAND                CREATED        STATUS        PORTS
6f350ef8d2ea   hyperledger/fabric-orderer:latest   "orderer"            "peer node start"      2 seconds ago Up Less than a second    0.0.0.0:7050->7050/tcp, 0.0.0.0
:7053->7053/tcp, 0.0.0.0:9443->9443/tcp
241c0d32f79e   hyperledger/fabric-peer:latest      "peer node start"    "peer node start"      2 seconds ago Up Less than a second    0.0.0.0:7051->7051/tcp, 0.0.0.0
:9444->9444/tcp
4c5b4be93377   hyperledger/fabric-peer:latest      "peer node start"    "peer node start"      2 seconds ago Up Less than a second    0.0.0.0:9051->9051/tcp, 7051/tc
p, 0.0.0.0:9445->9445/tcp

```

Starting the test network in docker container

Wrote the code using nano editor

```
tannmayy14@TanmayLaptop:~/projects/fabric-samples/test-network$ go version
go version go1.22.2 linux/amd64
```

```
tannmayy14@TanmayLaptop:~/projects/fabric-samples/test-network/asset-transfer-go$ go mod init asset-transfer-go
go: creating new go.mod: module asset-transfer-go
go: to add module requirements and sums:
    go mod tidy
```

```
2026-03-01 20:31:58.665 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [a8af15c2194e404703901f4d906d65d63a88b27eb6af9c547ee2f7cef2d7dd39] committed with status (VALID) at localhost:7051
2026-03-01 20:31:58.677 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [a8af15c2194e404703901f4d906d65d63a88b27eb6af9c547ee2f7cef2d7dd39] committed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name asset
+ res=0
Committed chaincode definition for chaincode 'asset' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name asset
+ res=0
Committed chaincode definition for chaincode 'asset' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

The Hyperledger Fabric test network was successfully configured and deployed on a WSL-based Linux environment. The custom Go chaincode was packaged, installed, approved, and committed on the channel *mychannel*. Asset transactions were executed using peer CLI commands, demonstrating successful interaction with the world state database. The experiment validates the working of the Fabric chaincode lifecycle including packaging, installation, approval, and invocation.

```
tannmayy14@TanmayLaptop:~/projects/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem" \
-C mychannel \
-n asset \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
-c '{"function": "CreateAsset", "Args": ["asset1", "Tanmay", "221009"]}'
```

Creating my asset:

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem" \
-C mychannel \
-n asset \
--peerAddresses localhost:7051 \
--tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
-c '{"function": "CreateAsset", "Args": ["asset1", "Tanmay", "221009"]}'
```

```
2026-03-01 20:39:11.793 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
tannmayy14@TanmayLaptop:~/projects/fabric-samples/test-network$
```

2026-03-01 20:39:11.793 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200